Střední průmyslová škola a Gymnázium Na Třebešíně

MATURITNÍ PRÁCE S OBHAJOBOU



INTERAKTIVNÍ DIGITÁLNÍ POZNÁMKOVÝ BLOK S INTEGROVANÝM AI ASISTENTEM PRO VÝUKU

Vypracoval:	Kostiantyn Kudinov
Vedoucí:	Mgr. Jan Souhrada
Třída:	4.G
Školní rok:	2026 / 2027

Střední průmyslová škola a Gymnázium Na Třebešíně

příspěvková organizace hl. m. Prahy

ZADÁNÍ MATURITNÍ PRÁCE

Téma maturitní práce:	**Interaktivní digitální poznámkový blok s integrovaným AI asistentem pro výuku**
Cíl maturitní práce:	• **vývoj multiplatformní klientské aplikace (desktop/web) s plynulým vektorovým vykreslováním a správou sešitů** • **integrace AI asistenta schopného kontextového vysvětlování konceptů zachycených na plátně.** • **implementace modulu pro automatické generování testovacích a procvičovacích úloh z poznámek.**
Forma odevzdání:	Digitální soubor (.pdf), výtisk dle požadavků školy

Jméno a příjmení řešitele(ů):	**Kostiantyn Kudinov**
Studijní obor:	Gymnázium 79 – 41 – K/41
Třída:	4.G
Školní rok:	2026/2027
Vedoucí maturitní práce:	Mgr. Jan Souhrada
Oponent maturitní práce:	

Termín odevzdání maturitní práce:	**24. 3. 2027**
Zadání převzal(i):	
Datum převzetí zadání:	**14. 10. 2026**

Maturitní práce odevzdána v sekretariátu školy (datum, potvrzení, podpis):

Tento list se zadáním je nedílnou součástí odevzdané maturitní práce.

V Praze dne 12. 10. 2026

Ing. Jakub Koželuh
ředitel školy

Čestné prohlášení

Prohlašuji, že jsem maturitní práci na téma Interaktivní digitální poznámkový blok s integrovaným AI asistentem pro výuku vypracoval samostatně s využitím odborné literatury a pramenů uvedených v přiloženém seznamu využitých zdrojů.

V Praze dne:

Podpis autora

Poděkování

Děkuji tímto vedoucímu maturitní práce panu Mgr. Janu Souhradovi za cenné připomínky a odborné rady při vypracovávání.

Název práce:

Interaktivní digitální poznámkový blok s integrovaným AI asistentem pro výuku

Autor:

Kostiantyn Kudinov

Vedoucí práce:

Mgr. Jan Souhrada

Abstrakt

Tato maturitní práce se zabývá návrhem a implementací multiplatformního interaktivního digitálního poznámkového bloku a virtuální tabule (whiteboardu) s integrovaným asistentem založeným na umělé inteligenci. Cílem práce je vytvořit klientskou aplikaci fungující jak ve webovém prohlížeči, tak jako nativní desktopová aplikace s plynulým vektorovým vykreslováním, správou vícestránkových sešitů, kolaborativní synchronizací v reálném čase a modulem pro kontextové vysvětlování studijní látky i automatické generování procvičovacích úloh. Teoretická část práce rozebírá principy počítačové 2D grafiky, afinních transformací souřadnicových prostorů, algoritmů vyhlazování tahů, architekturu *local-first* softwaru s využitím bezkolidních replikovaných datových typů (CRDT) a principy velkých jazykových modelů (LLM) ve vzdělávání. Dále představuje použité technologie, zejména jazyky TypeScript a Rust, knihovnu React, framework Tauri, protokol WebSocket a databázový systém SQLite. Praktická část následně popisuje návrh architektury, implementaci jednotlivých podsystémů, optimalizaci vykreslovacího jádra a testování výsledné aplikace.

Klíčová slova

digitální poznámkový blok, interaktivní tabule, Tauri, Rust, React, TypeScript, HTML5 Canvas, CRDT, Yjs, umělá inteligence, vzdělávání

Summary

This graduation thesis deals with the design and implementation of a cross-platform interactive digital notebook and virtual whiteboard featuring an integrated artificial intelligence assistant for education. The aim of the thesis is to develop a client application operating both in a web browser and as a native desktop application, providing smooth vector rendering, multi-page notebook management, real-time collaborative synchronization, and a module for contextual concept explanation and automated practice exercise generation. The theoretical part examines the principles of 2D computer graphics, affine coordinate transformations, stroke smoothing algorithms, local-first software architecture utilizing Conflict-Free Replicated Data Types (CRDTs), and the foundations of Large Language Models (LLMs) in education. Furthermore, it introduces the technologies used, specifically TypeScript, Rust, React, the Tauri framework, the WebSocket protocol, and SQLite. The practical part describes the architectural design, subsystem implementation, rendering optimizations, and testing of the resulting application.

Keywords

digital notebook, interactive whiteboard, Tauri, Rust, React, TypeScript, HTML5 Canvas, CRDT, Yjs, artificial intelligence, education

Obsah

1 Úvod ..... - 9 -

1.1 Cíle práce ..... - 9 -

2 Teoretická část ..... - 10 -

2.1 Digitální poznámkové bloky a počítačová 2D grafika ..... - 10 -

2.1.1 Koncept nekonečného plátna a stránkovaného sešitu ..... - 10 -

2.1.2 Rastrová a vektorová reprezentace grafických dat ..... - 10 -

2.1.3 Souřadnicové prostory a afinní transformace pohledu ..... - 11 -

2.1.4 Hustota pixelů displeje (HiDPI) a vykreslování volné ruky ..... - 12 -

2.2 Architektura synchronizace a komunikace v reálném čase ..... - 12 -

2.2.1 Paradigma Local-First softwaru ..... - 13 -

2.2.2 Bezkolidní replikované datové typy (CRDT) a knihovna Yjs ..... - 13 -

2.2.3 Protokol WebSocket ..... - 14 -

2.3 Umělá inteligence a velké jazykové modely ve vzdělávání ..... - 14 -

2.3.1 Architektura Transformer a multimodální jazykové modely ..... - 15 -

2.3.2 Pedagogické principy AI asistentů při výuce ..... - 15 -

2.4 Použité programovací jazyky a kmenové technologie ..... - 16 -

2.4.1 TypeScript a JavaScript ..... - 16 -

2.4.2 Knihovna React ..... - 16 -

2.4.3 Programovací jazyk Rust ..... - 17 -

2.4.4 Framework Tauri a srovnání s alternativami ..... - 17 -

2.4.5 Serverové prostředí Node.js, Hono a databáze SQLite ..... - 18 -

2.5 Vývojové nástroje a pomocné knihovny ..... - 19 -

2.5.1 Sestavovací nástroj Vite a stylovací framework Tailwind CSS ..... - 19 -

2.5.2 Knihovna PDF.js ..... - 19 -

2.5.3 Visual Studio Code, Git a GitHub ..... - 20 -

3 Praktická část ..... - 21 -

3.1 Architektura a struktura projektu ..... - 21 -

3.2 Implementace vykreslovacího jádra a správy sešitů ..... - 21 -

3.3 Nativní vrstva v prostředí Tauri a Rust ..... - 21 -
3.4 Kolaborativní synchronizace a serverová část ..... - 21 -
3.5 Integrace AI asistenta a generování úloh ..... - 21 -
3.6 Testování a optimalizace aplikace ..... - 21 -
4 Závěr ..... - 22 -
5 Seznam obrázků ..... - 23 -
6 Seznam využitých zdrojů ..... - 24 -

1 Úvod

Digitalizace vzdělávání a každodenního studia přinesla v posledních letech zásadní změnu ve způsobu, jakým studenti i pedagogové zaznamenávají a zpracovávají informace. Zatímco pro psaní lineárního textu existuje řada vyspělých textových editorů, při studiu exaktních a technických oborů – jako je matematika, fyzika, informatika či inženýrské disciplíny – je nezbytné volné prostorové uspořádání poznámek, ruční kreslení schémat, zapisování vzorců a anotování studijních materiálů ve formátu PDF.

Stávající řešení pro digitální psaní poznámek však často trpí zásadními kompromisy. Řada moderních multiplatformních aplikací je postavena na technologiích, které s sebou přinášejí vysokou paměťovou náročnost a znatelnou vstupní odezvu (latenci) při kreslení v prostředí linuxových i dalších desktopových systémů. Zároveň je využití nástrojů umělé inteligence při studiu obvykle odděleno do samostatných konverzačních oken ve webovém prohlížeči. Student tak musí své ručně psané výpočty či poznámky složitě přepisovat nebo snímkovat a přenášet mimo prostředí svého sešitu, čímž dochází ke ztrátě kontextu a plynulosti učení.

Motivací této maturitní práce je překonat uvedená omezení a vytvořit ucelený, vysoce výkonný interaktivní digitální poznámkový blok, který spojuje plynulé vektorové vykreslování na nekonečném i stránkovaném plátně s nativní systémovou integrací, synchronizací v reálném čase a vestavěným AI asistentem pracujícím přímo s kontextem poznámek na plátně.

1.1 Cíle práce

V souladu se zadáním maturitní práce byly stanoveny tři hlavní cíle:

- **Vývoj multiplatformní klientské aplikace (desktop/web) s plynulým vektorovým vykreslováním a správou sešitů:** Navrhnout a implementovat aplikaci běžící jak v prostředí webového prohlížeče, tak jako nativní desktopová aplikace s hardwarově akcelerovaným vykreslováním tahů bez vstupní prodlevy, podporou nekonečného plátna i stránkovaných sešitů (včetně importu dokumentů PDF) a perzistentní historií úprav.
- **Integrace AI asistenta schopného kontextového vysvětlování konceptů zachycených na plátně:** Propojit prostředí poznámkového bloku s multimodálním jazykovým modelem tak, aby uživatel mohl označit libovolnou oblast svých ručně psaných či kreslených poznámek a získat srozumitelné vysvětlení, doplnění teorie nebo kontrolu postupu řešení.
- **Implementace modulu pro automatické generování testovacích a procvičovacích úloh z poznámek:** Vytvořit mechanismus, který na základě obsahu sešitu vygeneruje interaktivní procvičovací otázky a úlohy pro aktivní upevnění studované látky.

2 Teoretická část

Tato kapitola vymezuje teoretická východiska, matematické principy a softwarové architektury, na nichž je postaven návrh a implementace interaktivního digitálního poznámkového bloku. V první části jsou rozebrány koncepty uživatelských rozhraní pro prostorovou práci s informacemi a matematické základy dvourozměrné počítačové grafiky. Následuje rozbor distribuované synchronizace dat v reálném čase, teorie velkých jazykových modelů ve vzdělávání a přehled použitých programovacích jazyků, knihoven a vývojových nástrojů.

2.1 Digitální poznámkové bloky a počítačová 2D grafika

Moderní aplikace pro digitální zaznamenávání poznámek a vizuální spolupráci představují syntézu dvou odlišných paradigmat interakce člověka s počítačem (HCI – *Human-Computer Interaction*): neomezeného prostorového plátna a strukturovaného stránkovaného dokumentu.

2.1.1 Koncept nekonečného plátna a stránkovaného sešitu

Koncept tzv. nekonečného plátna (*infinite canvas*) vychází z teorie přibližovatelných uživatelských rozhraní (ZUI – *Zoomable User Interface*), jejíž základy položili Benjamin B. Bederson a James D. Hollan v rámci systému Pad++ [1]. V architektuře ZUI není pracovní plocha omezena pevnými rozměry obrazovky ani jednorozměrným vertikálním posuvníkem, ale je definována jako souvislá dvourozměrná eukleidovská rovina $\mathbb{R}^2$. Uživatel se v tomto prostoru pohybuje pomocí plynulého posunu (*panning*) a změny měřítka (*zooming*), což umožňuje organizovat informace prostorově do myšlenkových map, architektonických diagramů a rozsáhlých výpočtů [1].

Naproti tomu strukturovaný stránkovaný sešit (*notebook mode*) zachovává metaforu fyzického papíru, která je klíčová pro lineární vedení školních zápisků, práci s učebními materiály a následný tisk či export do formátu PDF. Standardizované rozměry stránek v digitální podobě vycházejí z mezinárodní normy ISO 216, která definuje řadu formátů A (včetně formátu A4 o rozměrech 210×297 mm) na základě konstantního poměru stran $1 : \sqrt{2} \approx 1 : 1{,}4142$ [2]. Tento tzv. Lichtenbergův poměr zaručuje, že při půlení či škálování listu zůstává geometrická podobnost stránky zachována, což usnadňuje přepočet rozvržení na obrazovkách s různým rozlišením [2].

2.1.2 Rastrová a vektorová reprezentace grafických dat

Při návrhu vykreslovacího jádra digitální tabule je zásadní volba vnitřní reprezentace nakreslených tahů. V počítačové grafice se rozlišují dva základní přístupy: rastrová (bitmapová) a vektorová grafika [3].

Rastrová grafika ukládá obraz jako obdélníkovou matici diskrétních obrazových bodů (pixelů), z nichž každý nese informaci o barvě a průhlednosti (např. v barevném modelu RGBA). Ačkoliv

je rastrový zápis vhodný pro fotografie nebo rastrované předlohy, při použití v interaktivním poznámkovém bloku vykazuje výrazné nevýhody:

- **Závislost na rozlišení:** Při přiblížení pohledu (*zoom-in*) nad 100 % dochází k viditelné pixelizaci (aliasingu) a ztrátě ostrosti hran [3].
- **Nemožnost zpětné manipulace s objekty:** Jakmile je tah zakreslen do rastrové matice, splývá s okolními pixely a nelze jej dodatečně vybrat jako samostatný objekt, přesunout ani změnit jeho barvu či tloušťku.
- **Vysoké paměťové nároky na velkém plátně:** Udržování rastrové vyrovnávací paměti pro rozsáhlé nebo nekonečné plátno by vyžadovalo alokaci gigabajtů grafické paměti.

Vektorová grafika naproti tomu popisuje obraz pomocí matematických a geometrických primitiv – uspořádaných množin bodů, úseček, polygonů a parametrických křivek [3]. Každý tah perem je v paměti uložen jako posloupnost diskrétních vzorků polohy a přítlaku:

$$S = \{(x_i, y_i, p_i) \mid i = 1, 2, \ldots, n\} \tag{1}$$

kde $(x_i, y_i) \in \mathbb{R}^2$ jsou souřadnice bodu ve virtuálním prostoru plátna a $p_i \in \langle 0, 1 \rangle$ vyjadřuje normalizovanou sílu přítlaku stylusu [4]. Samotná rasterizace (převod vektorového popisu na pixely obrazovky) probíhá až v okamžiku vykreslování snímku pro aktuální úroveň přiblížení. Díky tomu zůstávají tahy dokonale ostré při libovolném měřítku a každý tah lze v datovém modelu nezávisle indexovat, transformovat, mazat či synchronizovat po síti [3, 5].

2.1.3 Souřadnicové prostory a afinní transformace pohledu

Aby mohl uživatel plynule posouvat pohled po plátně a měnit úroveň přiblížení, musí vykreslovací jádro neustále převádět souřadnice mezi dvěma souřadnicovými systémy [3]:

1. **Souřadnicový systém obrazovky (*Screen Space*):** Počátek $(0, 0)$ leží v levém horním rohu okna prohlížeče či nativní aplikace; jednotkou jsou logické CSS pixely, v nichž operační systém předává události ukazatele (myši, dotyku či stylusu) [4].
2. **Souřadnicový systém světa (*World Space*):** Virtuální nekonečná rovina plátna, v níž jsou uloženy absolutní souřadnice všech stránek, vektorových tahů a vložených obrázků.

Vztah mezi těmito prostory je dán afinní transformací skládající se ze změny měřítka (škálování koeficientem $s > 0$) a posunu (translace o vektor (t_x, t_y)) [3, 5]. V homogenních souřadnicích lze přechod ze souřadnic světa (x_w, y_w) do souřadnic obrazovky (x_s, y_s) zapsat maticově:

$$\begin{pmatrix} x_s \\ y_s \\ 1 \end{pmatrix} = \begin{pmatrix} s & 0 & t_x \\ 0 & s & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_w \\ y_w \\ 1 \end{pmatrix} = \begin{pmatrix} s \cdot x_w + t_x \\ s \cdot y_w + t_y \\ 1 \end{pmatrix} \tag{2}$$

Při záznamu nového tahu ukazatelem nebo při testování výběru objektů (*hit-testing*) je nutné aplikovat inverzní transformaci, která převede pozici kurzoru na obrazovce (x_s, y_s) zpět do souřadnic světa (x_w, y_w):

$$x_w = \frac{x_s - t_x}{s}, \qquad y_w = \frac{y_s - t_y}{s} \tag{3}$$

Při přibližování pohledu kolečkem myši či gestem nad konkrétním bodem obrazovky (x_p, y_p) je navíc žádoucí, aby bod světa pod kurzorem zůstal na obrazovce na stejném místě. Pokud se měřítko změní z původní hodnoty s_1 na novou hodnotu s_2, nový posun kamery (t'_x, t'_y) se vypočítá ze vztahu [3]:

$$t'_x = x_p - (x_p - t_x) \cdot \frac{s_2}{s_1}, \qquad t'_y = y_p - (y_p - t_y) \cdot \frac{s_2}{s_1} \tag{4}$$

2.1.4 Hustota pixelů displeje (HiDPI) a vykreslování volné ruky

Moderní monitory a přenosné počítače (označované jako HiDPI nebo Retina) využívají vysokou hustotu fyzických obrazových bodů. Webové standardy proto rozlišují mezi logickým CSS pixelem a fyzickým pixelem zobrazovacího panelu. Jejich poměr udává vlastnost `window.devicePixelRatio` (DPR) [6]:

$$\text{DPR} = \frac{\text{rozlišení ve fyzických pixelech}}{\text{rozlišení v logických CSS pixelech}} \tag{5}$$

Pokud má prvek `<canvas>` nastavenu logickou velikost v CSS na $W \times H$ pixelů, ale jeho vnitřní rastrová vyrovnávací paměť (*backing store*) není vynásobena koeficientem DPR, prohlížeč vykreslí scénu v nízkém rozlišení a následně ji lineárně roztáhne, což způsobuje rozmazání obrazu [6]. Správná inicializace proto vyžaduje nastavení vnitřní šířky a výšky plátna na $\lfloor W \cdot \text{DPR} \rfloor \times \lfloor H \cdot \text{DPR} \rfloor$ a současné vynásobení transformační matice kontextu koeficientem DPR [5, 6].

Pro samotné vykreslení tahu proměnlivé tloušťky (simulující reálné pero či štětec) nestačí vykreslit prostou lomenou čáru konstantní šířky. V moderních grafických aplikacích se využívá algoritmus teselace obrysového polygonu, jehož referenční implementaci představuje knihovna `perfect-freehand` autora Steva Ruize [7]. Algoritmus přijímá posloupnost vstupních bodů (x_i, y_i, p_i), aplikuje vyhlazení trajektorie (*streamline interpolation*), v každém bodě vypočítá normálový vektor kolmý na směr pohybu pera a ve vzdálenosti úměrné přítlaku p_i (nebo rychlosti pohybu ukazatele) zkonstruuje levou a pravou hranici tahu. Výsledný uzavřený polygon je následně proložen kvadratickými Bézierovými křivkami a vyplněn na grafickém plátně, což vytváří přirozený a plynulý rukopis [7].

2.2 Architektura synchronizace a komunikace v reálném čase

Interaktivní poznámkový blok klade vysoké nároky na správu stavu: musí fungovat okamžitě bez závislosti na kvalitě internetového připojení a zároveň umožňovat synchronizaci poznámek mezi

více zařízeními či spolupracujícími uživateli v reálném čase.

2.2.1 Paradigma Local-First softwaru

Tradiční webové aplikace využívají centralizovaný model klient–server, v němž je primárním zdrojem pravdy vzdálená databáze na serveru. Pokud uživatel provede úpravu, klient odešle požadavek na server a čeká na potvrzení; při výpadku připojení přestává být aplikace funkční.

Alternativu představuje architektura *Local-First Software*, kterou v roce 2019 formálně definovali Martin Kleppmann a kolektiv z výzkumné laboratoře Ink & Switch [8]. V tomto modelu jsou data primárně uložena a zpracovávána přímo v úložišti klientského zařízení, zatímco vzdálený server slouží k zálohování a distribuci změn mezi ostatní klienty. Architektura *local-first* přináší pro poznámkový blok tři zásadní vlastnosti [8]:

1. **Nulová síťová latence při interakci:** Čtení i zápis probíhají přímo v lokální paměti a lokálním úložišti prohlížeče (`localStorage` a `IndexedDB`) [9, 10], takže odezva uživatelského rozhraní při kreslení nezávisí na odezvě sítě.
2. **Plnohodnotný režim offline:** Uživatel může vytvářet a upravovat sešity bez přístupu k internetu; provedené změny se synchronizují automaticky po obnovení spojení.
3. **Vlastnictví a trvanlivost dat:** Uživatelská data zůstávají fyzicky dostupná na zařízení uživatele a lze je kdykoliv exportovat do otevřeného souborového formátu [8].

2.2.2 Bezkolidní replikované datové typy (CRDT) a knihovna Yjs

Pokud více klientů upravuje stejný dokument současně (nebo pokud jeden uživatel provede změny offline a následně se připojí k síti), vzniká problém souběžných aktualizací a řešení konfliktů. Historicky se v kolaborativních editorech používala metoda operační transformace (OT – *Operational Transformation*), která však vyžaduje centrální koordinační server a je náchylná k chybám při složitých souběžných operacích [8].

Moderním řešením tohoto problému jsou **bezkolidní replikované datové typy** (CRDT – *Conflict-Free Replicated Data Types*), jejichž matematický model formalizovali Marc Shapiro, Nuno Preguiça, Carlos Baquero a Marek Zawirski v roce 2011 [11]. Datové struktury CRDT zaručují vlastnost označovanou jako **silná konečná konzistence** (*Strong Eventual Consistency* – SEC):

- Každá lokální replika může být upravována okamžitě bez nutnosti žádat o zámek (*lock*) nebo čekat na souhlas ostatních uzlů.
- Operace slučování stavů (tzv. *merge* u stavově orientovaných CvRDT) nebo aplikování aktualizací (u operačně orientovaných CmRDT) je matematicky komutativní, asociativní a idempotentní [11].

- Jakmile si libovolné dvě repliky vymění stejnou množinu aktualizací (bez ohledu na pořadí jejich doručení po síti), dospějí deterministicky k identickému vnitřnímu stavu bez ztráty dat a bez nutnosti manuálního řešení konfliktů [11].

V prostředí jazyka JavaScript a TypeScript je nejrozšířenější vysoce optimalizovanou implementací CRDT knihovna **Yjs**, kterou vytvořil Kevin Jahns [12]. Yjs reprezentuje sdílený dokument (`Y.Doc`) jako kolekci sdílených datových typů, jako jsou `Y.Array` (pro uspořádané seznamy tahů, obrázků či stránek) a `Y.Map` (pro klíč–hodnota metadata dokumentu). Každá vkládaná položka je interně uložena jako struktura `Item` s unikátním identifikátorem dvojice $(\texttt{clientID}, \texttt{clock})$, kde `clock` je logické Lamportovo časové razítko. Při synchronizaci mezi klienty a serverem si uzly vyměňují kompaktní binárně kódované stavové vektory (*state vectors*) a rozdílové aktualizace (*diffs*), což minimalizuje přenášený objem dat po síti [12].

2.2.3 Protokol WebSocket

Pro obousměrný přenos binárních aktualizací CRDT a telemetrie kurzorů v reálném čase není klasický protokol HTTP vhodný, neboť je založen na modelu dotaz–odpověď (*request–response*) inicovaném výhradně klientem. Pro překonání tohoto omezení byl organizací IETF v roce 2011 standardizován protokol **WebSocket** v dokumentu RFC 6455 [13].

Protokol WebSocket poskytuje plně duplexní (*full-duplex*), obousměrný komunikační kanál přes jediné trvalé spojení TCP [13]. Navázání spojení začíná standardním požadavkem HTTP `GET` obsahujícím hlavičky `Upgrade: websocket` a `Connection: Upgrade`. Pokud server přechod potvrdí stavovým kódem `101 Switching Protocols`, protokoly se přepnou z HTTP na binární rámcový protokol WebSocket [13].

Oproti opakovanému dotazování (*HTTP polling*) přináší WebSocket minimální režii hlaviček (pouze 2 až 14 bajtů na rámec) a umožňuje serveru okamžitě odesílat (*push*) změny všem připojeným klientům v milisekundových intervalech. Protokol nativně rozlišuje mezi textovými rámci (kódovanými v UTF-8, vhodnými pro JSON zprávy o pohybu kurzorů) a binárními rámci (`ArrayBuffer` / `Uint8Array`, vhodnými pro přenos komprimovaných aktualizací Yjs CRDT) [13].

2.3 Umělá inteligence a velké jazykové modely ve vzdělávání

Integrace asistenta umělé inteligence do studijního poznámkového bloku vychází z pokroků v oblasti zpracování přirozeného jazyka (NLP – *Natural Language Processing*), počítačového vidění a multimodálních neuronových sítí.

2.3.1 Architektura Transformer a multimodální jazykové modely

Základním stavebním kamenem současných velkých jazykových modelů (LLM – *Large Language Models*) je architektura neuronové sítě **Transformer**, kterou v roce 2017 představil tým výzkumníků pod vedením Ashishe Vaswaniho v práci *Attention Is All You Need* [14]. Na rozdíl od starších rekurentních neuronových sítí (RNN a LSTM), které zpracovávaly vstupní sekvenci tokenů striktně sekvenčně zleva doprava, využívá Transformer mechanismus tzv. **sebe-pozornosti** (*self-attention*).

Mechanismus pozornosti počítá pro každý vstupní prvek (token) vážené vazby ke všem ostatním prvkům v kontextovém okně současně pomocí matic dotazů (Q – *Queries*), klíčů (K – *Keys*) a hodnot (V – *Values*) podle vztahu [14]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{6}$$

kde d_k je dimenze vektorů klíčů a funkce softmax převádí skóre podobnosti na pravděpodobnostní rozdělení. Tento přístup umožnil masivní paralelizaci tréninku na grafických akcelerátorech a schopnost zachytit dlouhodobé sémantické souvislosti v textu i matematickém zápisu [14].

Pro práci s ručně psanými poznámkami a diagramy na digitálním plátně mají klíčový význam **multimodální modely** (VLM – *Vision-Language Models*), které propojují vizuální enkodér obrazu s jazykovým dekodérem do společného vektorového prostoru příznaků (*embedding space*). Model tak dokáže analyzovat přímo rastrovaný výřez plátna obsahující ručně psané rovnice, grafy funkcí nebo vývojové diagramy společně s textovým dotazem uživatele.

2.3.2 Pedagogické principy AI asistentů při výuce

Využití velkých jazykových modelů ve vzdělávání spadá do oblasti inteligentních výukových systémů (ITS – *Intelligent Tutoring Systems*). Jak uvádějí Enkelejda Kasneci a kolektiv ve studii publikované v časopise *Learning and Individual Differences* [15], hlavní přínos LLM pro studenty spočívá v individualizaci výuky a okamžité zpětné vazbě.

V kontextu interaktivního poznámkového bloku plní AI asistent dvě klíčové didaktické funkce [15]:

1. **Kontextové lešení (*Instructional Scaffolding*) a vysvětlování konceptů:** Namísto poskytování obecných encyklopedických definic vychází asistent z konkrétního zápisu či náčrtu, který student právě vytvořil na plátně. Dokáže tak identifikovat konkrétní krok ve výpočtu, kterému student nerozumí, vysvětlit jej krok za krokem a upozornit na případné logické či numerické chyby [15].

2. **Aktivní vybavování (*Active Recall*) a generování procvičovacích úloh:** Výzkumy kognitivní psychologie dlouhodobě prokazují, že pasivní čtení poznámek je výrazně méně efektivní než aktivní testování znalostí. Automatické generování testovacích otázek a příkladů

přímo z obsahu studijního sešitu umožňuje studentovi okamžitě ověřit míru porozumění probíranému tématu [15].

2.4 Použité programovací jazyky a kmenové technologie

Pro realizaci multiplatformní architektury poznámkového bloku byla zvolena kombinace moderních webových a systémových technologií, které propojují rychlost vývoje uživatelského rozhraní s výkonem nativního kódu.

2.4.1 TypeScript a JavaScript

TypeScript je staticky typovaný programovací jazyk vyvíjený společností Microsoft od roku 2012, který představuje syntaktickou nadmnožinu jazyka JavaScript (standardizovaného pod označením ECMAScript) [16]. Veškerý platný kód v jazyce JavaScript je současně platným kódem v jazyce TypeScript, avšak TypeScript rozšiřuje jazyk o statický typový systém, rozhraní (`interface`), generické typy, výčtové typy a typovou inferenci [16].

Při sestavování projektu (*transpilaci*) provádí překladač TypeScriptu důkladnou statickou analýzu kódu a následně odstraní typové anotace, čímž vygeneruje čistý JavaScript spustitelný v libovolném webovém prohlížeči nebo v prostředí Node.js. V rozsáhlé grafické aplikaci zabraňuje statické typování celé třídě běhových chyb, neboť vynucuje přesnou strukturu souřadnicových bodů, vektorových tahů, stránek sešitu, stavů historie (Undo/Redo) i síťových zpráv [16].

2.4.2 Knihovna React

React je open-source knihovna pro tvorbu uživatelských rozhraní, původně vyvinutá Jordanem Walkem ve společnosti Meta (dříve Facebook) a uvolněná v roce 2013 [17]. React je založen na **deklarativním a komponentovém přístupu**: uživatelské rozhraní je rozděleno do samostatných, znovupoužitelných komponent, z nichž každá spravuje svůj vlastní lokální stav a popisuje, jak má rozhraní v daném stavu vypadat [17].

Pro řízení stavu a životního cyklu funkcionálních komponent využívá React systém tzv. **Hooků** (*Hooks*) [17]:

- `useState` slouží k uchovávání reaktivního stavu komponenty (např. aktivní kreslicí nástroj, zvolená barva pera, otevřená modální okna), jehož změna vyvolá překreslení rozhraní.
- `useEffect` umožňuje synchronizovat komponentu s externími systémy (např. navázání spojení WebSocket, registrace globálních klávesových zkratek či naslouchání změnám velikosti okna přes `ResizeObserver`).
- `useRef` uchovává mutovatelnou referenci, která přetrvává mezi jednotlivými vykresleními

bez vyvolávání reaktivního přepočtu virtuálního DOMu (*Virtual DOM*) [17].

Kombinace deklarativního Reactu pro ovládací panely (lišty nástrojů, správa sešitů, panel AI asistenta) a imperativního přístupu přes `useRef` pro přímé vykreslování na prvky `<canvas>` ve smyčce `requestAnimationFrame` je klíčová pro výkon aplikace: během pohybu pera po plátně nedochází k rekonciliaci stromu React komponent, takže vykreslování dosahuje plné obnovovací frekvence monitoru [5, 17].

2.4.3 Programovací jazyk Rust

Rust je kompilovaný, staticky typovaný systémový programovací jazyk navržený s důrazem na vysoký výkon, souběžnost a paměťovou bezpečnost [18]. Původně jej vytvořil Graydon Hoare ve společnosti Mozilla Research a první stabilní verze Rust 1.0 byla vydána v roce 2015.

Na rozdíl od jazyků jako C nebo C++, v nichž manuální správa paměti často vede k závažným chybám (přístup k uvolněné paměti *use-after-free*, přetečení vyrovnávací paměti či souběhy dat *data races*), a na rozdíl od jazyků jako Java, C# či Go, které k uvolňování paměti využívají běhový sběrač nepoužívané paměti (*Garbage Collector* – GC), zavádí Rust unikátní model **vlastnictví (*ownership*) a vypůjčování (*borrowing*)** kontrolovaný přímo v době kompilace tzv. *borrow checkerem* [18]:

1. Každá hodnota v paměti má právě jednoho vlastníka (*owner*).
2. V jednom okamžiku může existovat buď libovolný počet neměnných referencí (`&T`), nebo právě jedna měnitelná reference (`&mut T`).
3. Jakmile vlastník opustí svůj platný rozsah (*scope*), paměť je automaticky a deterministicky uvolněna bez režie GC [18].

Díky přímé kompilaci do nativního strojového kódu prostřednictvím backendu LLVM dosahuje Rust výkonu srovnatelného s jazykem C++, což z něj činí ideální jazyk pro nativní jádro desktopové aplikace [18].

2.4.4 Framework Tauri a srovnání s alternativami

Tauri je open-source framework pro tvorbu multiplatformních desktopových a mobilních aplikací, jehož stabilní verze Tauri 2.0 představuje moderní alternativu k tradičnímu frameworku Electron [19].

Zatímco aplikace postavené na frameworku Electron přibalují ke každému programu kompletní instanci prohlížeče Chromium a běhové prostředí Node.js (což vede k instalačním balíčkům o velikosti přes 150 MB a vysoké spotřebě paměti RAM), Tauri odděluje aplikační jádro v jazyce Rust od zobrazovací vrstvy, pro kterou využívá **nativní systémový WebView** již přítomný v operačním systému [19]:

- **Linux:** knihovna `WebKitGTK` (součást grafického prostředí GTK) spolupracující se zobrazovacím protokolem Wayland nebo X11 [19, 20],
- **Windows:** komponenta `Microsoft Edge WebView2` (založená na jádře Chromium),
- **macOS:** systémový framework `WKWebView` (založený na jádře WebKit) [19].

Komunikace mezi frontendovou částí (TypeScript/React) a nativním jádrem (Rust) probíhá prostřednictvím asynchronního meziprocesového mostu (IPC – *Inter-Process Communication*). Funkce v jazyce Rust označené atributovým makrem `#[tauri::command]` lze z frontendu bezpečně volat pomocí funkce `invoke()`, přičemž parametry i návratové hodnoty jsou automaticky serializovány pomocí knihovny `serde` do formátu JSON [19]. Tauri navíc uplatňuje přísný bezpečnostní model založený na oprávněních (*Capabilities / ACL*), který přesně vymezuje, ke kterým systémovým příkazům má webové rozhraní přístup [19]. Srovnání obou architektur shrnuje Tab. 1.

Parametr	**Framework Tauri v2**	**Framework Electron**
Nativní backend	Kompilovaný jazyk Rust	Interpretovaný Node.js (C++/JS)
Vykreslovací jádro	Systémové (WebKitGTK, WebView2, WKWebView)	Přibalená instance prohlížeče Chromium
Velikost instalátoru	Typicky 3 až 10 MB	Typicky 120 až 250 MB
Spotřeba paměti RAM	Nízká (sdílené systémové knihovny)	Vysoká (samostatný proces prohlížeče)
Bezpečnostní model	Granulární systém Capabilities (ACL)	Širší přístup k API prostředí Node.js

Tab. 1: Architektonické srovnání frameworků Tauri v2 a Electron [19]

2.4.5 Serverové prostředí Node.js, Hono a databáze SQLite

Pro zajištění centrální synchronizace, trvalého ukládání sdílených sešitů a distribuce webové verze aplikace je využit serverový stack postavený na prostředí Node.js, frameworku Hono a databázi SQLite:

- **Node.js:** Asynchronní, událostmi řízené (*event-driven*) běhové prostředí pro JavaScript a TypeScript mimo prohlížeč, které v roce 2009 vytvořil Ryan Dahl nad vysoce výkonným kompilátorem Google V8 [21]. Díky neblokujícímu modelu vstupně-výstupních operací (non-blocking I/O) a jednovláknové smyčce událostí (*event loop*) dokáže Node.js efektivně obsluhovat stovky souběžných WebSocket spojení s minimální režií [21].

- **Hono:** Moderní, ultrarychlý webový framework pro TypeScript postavený přímo na standardizovaných Web API (`Request`, `Response`, `Fetch`) [22]. Slouží ke směrování požadavků REST API, obsluze statických souborů sestavené aplikace a integraci se serverem WebSocket.

- **SQLite a režim WAL:** SQLite je vestavěný (*embedded*) relační databázový systém vytvořený D. Richardem Hippem v roce 2000, který ukládá celou databázi včetně indexů a tabulek do jediného multiplatformního souboru na disku bez nutnosti provozovat samostatný databázový proces [23]. Při aktivaci žurnálovacího režimu **WAL** (*Write-Ahead Logging*) nejsou změny zapisovány přímo do hlavního databázového souboru, ale jsou nejprve sekvenčně připojovány do samostatného souboru `-wal`. To umožňuje souběžné čtení databáze během probíhajícího zápisu a výrazně zvyšuje propustnost při častém ukládání stavů plátna [23].

2.5 Vývojové nástroje a pomocné knihovny

Při vývoji, sestavování a správě projektu byla využita sada specializovaných knihoven a inženýrských nástrojů, které zajišťují modularitu kódu a efektivní vývojový cyklus.

2.5.1 Sestavovací nástroj Vite a stylovací framework Tailwind CSS

Vite je moderní sestavovací nástroj (*build tool*) a vývojový server pro webové aplikace, který vytvořil Evan You [24]. Během vývoje využívá Vite nativní podporu modulů ECMAScript (ESM) přímo v prohlížeči a překládá zdrojové soubory TypeScriptu a JSX na vyžádání. Díky mechanismu *Hot Module Replacement* (HMR) se úpravy kódu projeví v běžící aplikaci během desítek milisekund bez ztráty stavu plátna. Pro produkční nasazení využívá Vite optimalizovaný bundler, který zdrojový kód minifikuje a rozdělí do statických balíčků [24].

Tailwind CSS je tzv. *utility-first* CSS framework, který namísto předdefinovaných vzhledů komponent poskytuje sadu atomických nízkoúrovňových tříd přímo pro rozvržení (`flex`, `grid`), rozestupy, typografii a barevná schémata [25]. Ve spojení s nativními CSS proměnnými (*CSS Custom Properties*) umožňuje konzistentní přepínání barevných režimů aplikace (světlý, teplý čtecí a tmavý režim) v reálném čase [25].

2.5.2 Knihovna PDF.js

Pro import studijních materiálů, pracovních listů a prezentací do prostředí poznámkového bloku je využita open-source knihovna **PDF.js** (`pdfjs-dist`), vyvíjená organizací Mozilla Foundation [26]. Knihovna implementuje parser a vykreslovací jádro standardu Portable Document Format (PDF) přímo v jazyce JavaScript. Pomocí asynchronního vlákna (*Web Worker*) dekóduje vnitřní strukturu dokumentu PDF a vykresluje jednotlivé stránky do rastrové podoby na prvky

HTML5 Canvas, které lze následně vložit jako podkladové listy sešitu a volně přes ně psát či kreslit [26].

2.5.3 Visual Studio Code, Git a GitHub

Jako hlavní vývojové prostředí byl zvolen **Visual Studio Code** (VS Code) od společnosti Microsoft [27]. Tento multiplatformní editor zdrojového kódu poskytuje prostřednictvím rozšíření plnou podporu pro jazykovou analýzu TypeScriptu, systémového jazyka Rust (`rust-analyzer`) i sazbu dokumentace v systému LaTeX [27].

Pro správu verzí zdrojového kódu byl využit distribuovaný verzovací systém **Git**, původně vytvořený Linusem Torvaldsem v roce 2005 pro potřeby vývoje jádra operačního systému Linux [28], v kombinaci s cloudovou platformou **GitHub**. Git uchovává historii projektu jako orientovaný acyklický graf snímků stavů (*commits*), což umožňuje bezpečné experimentování s novými funkcemi, sledování historie změn a zálohování celého projektu [28].

3 Praktická část

Tato kapitola popisuje samotný návrh architektury, implementaci jednotlivých modulů aplikace a řešení technických výzev, které se během vývoje vyskytly.

3.1 Architektura a struktura projektu

3.2 Implementace vykreslovacího jádra a správy sešitů

3.3 Nativní vrstva v prostředí Tauri a Rust

3.4 Kolaborativní synchronizace a serverová část

3.5 Integrace AI asistenta a generování úloh

3.6 Testování a optimalizace aplikace

4 Závěr

Závěr…

5 Seznam obrázků

6 Seznam využitých zdrojů

[1] BEDERSON, Benjamin B. a James D. HOLLAN. Pad++: A Zooming Graphical Interface for Exploring Alternate Interface Physics. In: *Proceedings of the 7th Annual ACM Symposium on User Interface Software and Technology (UIST '94)* [online]. New York: ACM, 1994, s. 17–26 [cit. 2026-10-19]. ISBN 0-89791-657-3. Dostupné z: https://doi.org/10.1145/192426.192435

[2] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *ISO 216:2007 Writing paper and certain classes of printed matter – Trimmed sizes – A and B series, and indication of machine direction* [online]. Geneva: ISO, 2007 [cit. 2026-10-19]. Dostupné z: https://www.iso.org/standard/36631.html

[3] MARSCHNER, Steve a Peter SHIRLEY. *Fundamentals of Computer Graphics*. 5th ed. Boca Raton: CRC Press, 2021. ISBN 978-0-367-50503-5.

[4] WORLD WIDE WEB CONSORTIUM (W3C). *Pointer Events – Level 3* [online]. W3C Working Draft, 2024 [cit. 2026-10-19]. Dostupné z: https://www.w3.org/TR/pointerevents3/

[5] MOZILLA CONTRIBUTORS. *Canvas API – Web APIs | MDN* [online]. San Francisco: Mozilla Foundation, 2026 [cit. 2026-10-19]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

[6] MOZILLA CONTRIBUTORS. *Window: devicePixelRatio property – Web APIs | MDN* [online]. San Francisco: Mozilla Foundation, 2026 [cit. 2026-10-19]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/Window/devicePixelRatio

[7] RUIZ, Steve. *perfect-freehand: Draw perfect pressure-sensitive freehand lines* [online]. GitHub, 2023 [cit. 2026-10-19]. Dostupné z: https://github.com/steveruizok/perfect-freehand

[8] KLEPPMANN, Martin, Adam WIGGINS, Peter VAN HARDENBERG a Mark MCGRANAGHAN. Local-first software: You own your data, in spite of the cloud. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)* [online]. New York: ACM, 2019, s. 154–178 [cit. 2026-10-19]. ISBN 978-1-4503-6995-4. Dostupné z: https://www.inkandswitch.com/local-first/

[9] WHATWG. *HTML Living Standard – Web Storage* [online]. 2026 [cit. 2026-10-19]. Dostupné z: https://html.spec.whatwg.org/multipage/webstorage.html

[10] WORLD WIDE WEB CONSORTIUM (W3C). *Indexed Database API 3.0* [online]. W3C Recommendation, 2025 [cit. 2026-10-19]. Dostupné z: https://www.w3.org/TR/IndexedDB/

[11] SHAPIRO, Marc, Nuno PREGUIÇA, Carlos BAQUERO a Marek ZAWIRSKI. Conflict-Free Replicated Data Types. In: *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)* [online]. Berlin: Springer, 2011, Lecture Notes in Computer Science, sv. 6976, s. 386–400 [cit. 2026-10-19]. ISBN 978-3-642-24549-7. Dostupné z: https://doi.org/10.1007/978-3-642-24550-3_29

[12] JAHNS, Kevin. *Yjs Documentation – Shared Editing & CRDT Framework* [online]. 2025 [cit. 2026-10-19]. Dostupné z: https://docs.yjs.dev/

[13] FETTE, Ian a Alexey MELNIKOV. *RFC 6455: The WebSocket Protocol* [online]. Internet Engineering Task Force (IETF), prosinec 2011 [cit. 2026-10-19]. Dostupné z: https://datatracker.ietf.org/doc/html/rfc6455

[14] VASWANI, Ashish et al. Attention Is All You Need. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)* [online]. Long Beach: Curran Associates, 2017, s. 5998–6008 [cit. 2026-10-19]. Dostupné z: https://arxiv.org/abs/1706.03762

[15] KASNECI, Enkelejda et al. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences* [online]. 2023, roč. 103, č. 102274 [cit. 2026-10-19]. ISSN 1041-6080. Dostupné z: https://doi.org/10.1016/j.lindif.2023.102274

[16] MICROSOFT CORPORATION. *TypeScript Documentation* [online]. Redmond: Microsoft, 2026 [cit. 2026-10-19]. Dostupné z: https://www.typescriptlang.org/docs/

[17] META PLATFORMS. *React Reference Overview* [online]. Menlo Park: Meta Platforms, 2026 [cit. 2026-10-19]. Dostupné z: https://react.dev/reference/react

[18] KLABNIK, Steve a Carol NICHOLS. *The Rust Programming Language* [online]. 2nd ed. San Francisco: No Starch Press, 2023 [cit. 2026-10-19]. ISBN 978-1-7185-0310-6. Dostupné z: https://doc.rust-lang.org/book/

[19] TAURI CONTRIBUTORS. *Tauri 2.0 Documentation* [online]. The Tauri Programme within The Commons Conservancy, 2026 [cit. 2026-10-19]. Dostupné z: https://v2.tauri.app/

[20] HØGSBERG, Kristian et al. *Wayland Protocol Specification* [online]. Freedesktop.org, 2025 [cit. 2026-10-19]. Dostupné z: https://wayland.freedesktop.org/docs/html/

[21] OPENJS FOUNDATION. *Node.js Documentation* [online]. 2026 [cit. 2026-10-19]. Dostupné z: https://nodejs.org/docs/latest/api/

[22] HONO CONTRIBUTORS. *Hono – Web framework built on Web Standards* [online]. 2026 [cit. 2026-10-19]. Dostupné z: https://hono.dev/docs/

[23] SQLITE CONSORTIUM. *Write-Ahead Logging in SQLite* [online]. Hipp, Wyrick & Company, 2026 [cit. 2026-10-19]. Dostupné z: https://www.sqlite.org/wal.html

[24] YOU, Evan a VITE CONTRIBUTORS. *Vite Guide & Documentation* [online]. 2026 [cit. 2026-10-19]. Dostupné z: https://vite.dev/guide/

[25] TAILWIND LABS. *Tailwind CSS Documentation (v4)* [online]. 2026 [cit. 2026-10-19]. Dostupné z: https://tailwindcss.com/docs

[26] MOZILLA FOUNDATION. *PDF.js – Portable Document Format Viewer in JavaScript* [online]. 2026 [cit. 2026-10-19]. Dostupné z: https://mozilla.github.io/pdf.js/

[27] MICROSOFT CORPORATION. *Visual Studio Code Documentation* [online]. Redmond: Microsoft, 2026 [cit. 2026-10-19]. Dostupné z: https://code.visualstudio.com/docs

[28] CHACON, Scott a Ben STRAUB. *Pro Git* [online]. 2nd ed. New York: Apress, 2014 [cit. 2026-10-19]. ISBN 978-1-4842-0077-3. Dostupné z: https://git-scm.com/book/en/v2